

Лабораторная работа №2. Форма, база данных и анализ эмоций

Что вы сделаете

Вы добавите на сайт первый работающий сервис по адресу `/services/emotions/`. Пользователь вводит текст, Django проверяет форму, локальная модель возвращает оценки эмоций, а результат и история сохраняются в SQLite только для текущей сессии браузера.

Главная тема работы — не устройство нейросети, а полный цикл Django: форма → view → модель базы данных → redirect → шаблон. Технический код нейросети уже дан в `services.py`; переписывать или заучивать его не нужно.

1. Выбор исходного проекта

Используйте один из вариантов:

1. Продолжите свой проект после ЛР1.
2. Если ЛР1 не завершена, распакуйте `starter.zip` из каталога ЛР2. Внутри находится папка `neuralnet` — готовое состояние после ЛР1 без базы, окружения и моделей.

Все следующие команды выполняйте в каталоге с `manage.py`. Если используете `starter`, сначала создайте и активируйте `.venv` по пункту 2 ЛР1.

2. Установка зависимостей

Активируйте окружение. Windows PowerShell:

```
.\.venv\Scripts\Activate.ps1
python -m pip install torch==2.14.0 --index-url https://download.pytorch.org/whl/cpu
python -m pip install Django==5.2.17 transformers==5.17.0
```

Windows cmd использует `.venv\Scripts\activate.bat`. Linux:

```
source .venv/bin/activate
python -m pip install torch==2.14.0 --index-url https://download.pytorch.org/whl/cpu
python -m pip install Django==5.2.17 transformers==5.17.0
```

macOS:

```
source .venv/bin/activate
python -m pip install Django==5.2.17 torch==2.14.0 transformers==5.17.0
```

Проверьте установку:

```
python -c "import django, torch, transformers; print(django.get_version())"
```

Команда должна вывести `5.2.17` без `traceback`.

3. Подключение готовых файлов сервиса

Распакуйте содержимое `готовые_файлы.zip` из каталога ЛР2 прямо в каталог с `manage.py`. Появятся:

```

text_classifier/
├── models.py          # структура записи в БД
├── core/fields.py     # совместимое хранение JSON-списков
├── forms.py           # форма и проверка длины
├── views.py           # обработка GET и POST
├── urls.py            # адрес страницы
├── services.py        # готовая обёртка нейросети
├── tests.py           # автоматические проверки
└── templates/text_classifier/index.html

```

Также добавится команда `core/management/commands/prepare_models.py`. Не выполняйте `startapp text_classifier`: приложение уже находится в архиве.

4. Регистрация приложения и адреса

В `config/settings.py` добавьте в конец `INSTALLED_APPS`:

```
"text_classifier",
```

В `config/urls.py` добавьте маршрут в `urlpatterns`:

```
path("services/emotions/", include("text_classifier.urls")),
```

Импорт в начале файла должен содержать `include`:

```
from django.urls import include, path
```

В `templates/base.html` внутри списка `.nav-list`, сразу после ссылки «Главная», добавьте:

```
<li><a href="{% url 'text_classifier:index' %}"
  {% if active_page == 'emotions' %}aria-current="page"{% endif %}>Эмоции</a></li>
```

В `core/views.py` найдите карточку «Анализ эмоций» и замените для неё:

```
"url_name": None,
```

на:

```
"url_name": "text_classifier:index",
```

Теперь и меню, и карточка ведут на один именованный маршрут. Имена удобнее жёстко записанного `/services/emotions/`: адрес можно изменить в одном месте.

Контрольная точка

```
python manage.py check
```

Если появилась ошибка `NoReverseMatch`, проверьте регистрацию приложения, маршрут и написание `text_classifier:index`.

5. Разбор модели данных

Откройте `text_classifier/models.py`. Одна запись `TextAnalysis` содержит:

- `session_key` — идентификатор сессии браузера;
- `text` — введённый текст, максимум 500 символов;
- `scores` — готовый список результатов в JSON;
- `created_at` — время создания.

Поле `session_key` не является учётной записью. Оно позволяет двум браузерам видеть разные истории без регистрации пользователей. Результаты хранятся в учебном `JSONListField`: в Python

это обычный список, а в SQLite — JSON-текст. Такой вариант работает даже в сборках SQLite без расширения JSON1. Пример значения:

```
[{"label": "Радость", "score": 91.2}]
```

Создайте и примените миграцию:

```
python manage.py makemigrations text_classifier
python manage.py migrate
```

`makemigrations` создаёт Python-описание изменения, а `migrate` применяет его к SQLite. Файл `db.sqlite3` создаётся локально и не должен попадать в отчёт или архив проекта.

6. Разбор формы

Откройте `text_classifier/forms.py`. `TextAnalysisForm` наследуется от `ModelForm`, поэтому Django строит форму по модели. В `fields` оставлено только `text`: пользователь не должен задавать `session_key`, результат или дату.

Обратите внимание на три уровня удобства:

- `label` говорит, что требуется ввести;
- `help_text` заранее сообщает ограничение;
- `maxlength=500` помогает браузеру, а `max_length=500` в модели обеспечивает серверную проверку.

Проверка на сервере обязательна: HTML можно обойти, отправив запрос не через обычную форму.

7. Разбор view по шагам

Откройте `text_classifier/views.py`. Функция `index` обрабатывает два случая.

При GET:

1. Получает или создаёт ключ сессии через `get_session_key(request)`.
2. Создаёт пустую форму.
3. Загружает только историю с текущим `session_key`.
4. Передаёт форму и историю шаблону.

При POST:

1. Создаёт `TextAnalysisForm(request.POST)`.
2. Вызывает `form.is_valid()`.
3. Передаёт очищенный текст готовому предиктору.
4. Вызывает `form.save(commit=False)`, чтобы перед сохранением дописать сессию и результат.
5. Сохраняет запись и делает `redirect` на её результат.

`Redirect` реализует схему Post/Redirect/Get. После успешной отправки обновление страницы повторяет GET, а не создаёт вторую запись.

Строка с `get_object_or_404(..., session_key=session_key)` важна: пользователь не сможет открыть чужой результат простой заменой номера в URL.

Про файл services.py

`services.py` принимает строку и возвращает список подписей и процентов. Модель загружается только при первом обращении и затем остаётся в памяти процесса. Для этой лабораторной достаточно знать вход и выход функции `predict`; математические детали не входят в контрольные вопросы.

Первую загрузку можно выполнить заранее:

```
python manage.py prepare_models --service text
```

Нужен интернет. Повторное скачивание обычно не требуется, потому что файлы сохраняются в пользовательском кеше.

8. Шаблон и корректные шкалы

Откройте `templates/text_classifier/index.html`. В нём есть форма, текущий результат и история. Проценты выводятся текстом, а ширина цветной шкалы вычисляется так:

```
{% widthratio item.score 100 100 as score_width %}  
<span style="width: {{ score_width }}%"></span>
```

`widthratio` превращает дробное значение в целое число для CSS. Это важно в русской локали: текстовое число может выглядеть как `12,5`, но CSS ожидает точку либо целое число. Поэтому шкала `12,5%` не должна заполняться целиком.

9. Запуск и ручная проверка

```
python manage.py runserver
```

Откройте `<http://127.0.0.1:8000/services/emotions/>` и проверьте:

1. Пустой текст показывает ошибку рядом с полем.
2. Обычная фраза создаёт результат; обновление страницы не добавляет копию.
3. Полосы имеют разную длину и примерно соответствуют показанным процентам.
4. В приватном окне история пуста.

Первый анализ может занять больше времени из-за загрузки модели. Смотрите сообщения в терминале и не запускайте сервер второй раз.

10. Автоматические тесты

```
python manage.py check  
python manage.py makemigrations --check --dry-run  
python manage.py test
```

Тесты заменяют тяжёлую модель маленьким объектом-заглушкой, поэтому не скачивают веса. Они проверяют форму, `redirect`, сохранение записи, изоляцию сессий и корректную ширину шкалы.

11. Две полезные UX-проверки

1. Дайте однопользователю ввести фразу и попросите объяснить результат. Должно быть понятно, что подпись и процент относятся к одной полосе и что это оценка программы, а не точный диагноз

текста.

2. Отправьте пустое поле, затем слишком длинный текст. Сообщение должно появиться рядом с формой и подсказывать, что исправить.

Запишите наблюдение и, если потребовалось, измените только понятный текст подписи или подсказки — логику нейросети менять не нужно.

12. Если что-то не работает

- `No module named text_classifier` — архив распакован не рядом с `manage.py` либо приложение не добавлено в `INSTALLED_APPS`.
- `no such table: text_classifier_textanalysis` — выполните обе команды миграций из пункта 5.
- Первый запрос долго выполняется — идёт скачивание; проверьте интернет и терминал или заранее запустите `prepare_models`.
- После POST запись создаётся дважды — убедитесь, что успешная ветка заканчивается `redirect`, а не `render`.
- История видна в другом браузере — во всех запросах к `TextAnalysis` должен быть фильтр `session_key=session_key`.
- Все полосы заполнены полностью — в шаблоне используйте `score_width` из пункта 8, а не дробное число напрямую.
- Порт занят — `python manage.py runserver 8001`.

13. Контрольные вопросы

1. Чем `ModelForm` упрощает работу с моделью Django?
2. Чем отличаются `makemigrations` и `migrate`?
3. Зачем после успешного POST выполняется `redirect`?
4. Почему ограничения HTML не заменяют серверную валидацию?
5. Для чего запросы к истории фильтруются по `session_key`?
6. Что делает `form.save(commit=False)`?

14. Критерии приёмки

- `/services/emotions/` открывается из меню и карточки.
- Валидный текст сохраняется и показывает результат, неверный ввод показывает ошибку.
- Шкалы визуально соответствуют процентам.
- Обновление результата не создаёт дубликат.
- Истории двух браузерных сессий не смешиваются.
- Все три проверочные команды завершаются успешно.
- Студент может объяснить модель, форму и последовательность действий view.

15. Краткий отчёт и защита

Сдайте DOCX или PDF: титульные данные; цель; перечень добавленных файлов; скриншоты формы, результата и ошибки; объяснение цепочки POST своими словами; результат двух UX-проверок; ответы на контрольные вопросы.

На защите запустите сервис, создайте результат и покажите `models.py`, `forms.py`, `views.py` и маршрут.